

软件过程与管理 — 个人小组工作总结

姓名：宋鑫锐

学号：2023141461180

课程：2026 春 — 软件过程与管理

小组项目：智慧商城电商平台（B2C SaaS）+ 超市货架YOLO-CLIP多模态问答系统

一、小组整体分工协作方式

1.1 团队组织结构

本小组共12名成员，按照项目模块和交付物类型划分为三个协作小组，采用"功能小组 × 交付物矩阵"的双维度协作模式：

项目经理（1人）

└─ 文档与过程组（4人） ─ 负责四份核心文档的撰写

└─ 成员A：01-AI技术应用场景匹配报告 主笔

└─ 成员B：02-软件过程改进方案设计文档 主笔

└─ 我（宋鑫锐）：04-改进后的软件过程定义文档 主笔 + 03-验证报告 协作

└─ 成员C：03-AI技术应用验证报告 主笔 + 01-场景匹配报告 协作

└─ 技术实现组（5人） ─ 负责YOLO-CLIP多模态问答系统开发

└─ 成员D：YOLO目标检测模块 + 视频处理管线

└─ 成员E：CLIP特征提取模块 + 模型下载与配置

└─ 成员F：FAISS向量索引模块 + 相似度验证脚本

└─ 我（宋鑫锐）：LLM智能问答模块 + Gradio交互界面集成

└─ 成员G：全栈联调 + 商业项目设计书撰写

└─ QA与集成组（2人）

└─ 成员H：测试验证 + 缺陷管理

└─ 成员I：CI/CD配置 + 文档质量审核

└─ PM（1人）：整体协调、进度跟踪、PPT制作

1.2 协作流程与工具链

协作维度	采用方式	工具/平台
代码版本管理	Git Flow分支策略（main/develop/feature）	GitHub
文档协作	异步协作 + 集中评审	Markdown + GitHub Wiki
沟通与同步	每日站会（线上15分钟）+ 每周全员评审会（线下2小时）	微信群 + 腾讯会议
AI工具使用	统一Prompt模板库 + 个人自主使用	Claude 4 / GPT-4o / Cursor
交付物管理	版本化文档 + 评审-修订-基线流程	GitHub Releases

1.3 我的双重角色

在本小组中，我承担了**跨文档与技术双线**的工作角色：

- **文档线**：作为04号文档（改进后的软件过程定义文档）的主笔，负责全文档框架设计、内容撰写、Mermaid流程图绘制、度量指标体系设计；同时参与03号验证报告中的测试验证数据分析部分
- **技术线**：负责YOLO-CLIP系统中的LLM智能问答模块（基于DeepSeek API的RAG问答）、Gradio交互界面开发，以及系统级联调

这种双线角色让我能够将技术实践中的一手经验直接反哺到过程文档的撰写中——例如在编写"AI辅助编码与审查"活动定义时，我基于自己在Gradio UI开发中实际使用Cursor的体验来校准AI效率提升数据。

二、个人参与完成的主要工作

2.1 文档工作（核心贡献）

2.1.1 04-改进后的软件过程定义文档 — 主笔（独立完成）

这是我投入时间最多、贡献最集中的交付物。该文档是小组作业中篇幅最长（约480行Markdown、9章完整结构）、综合性最强的核心文档。我的具体工作包括：

（1）整体框架设计

基于ISO/IEC 12207:2017的四大过程组（协议过程组、组织项目使能过程组、技术管理过程组、技术过程组）框架，设计了"标准框架保留 + AI节点嵌入"的文档结构。核心设计决策包括：

- 保留ISO/IEC 12207的完整过程分类体系，确保与前期作业的可追溯性
- 在每个过程定义中增加"AI参与节点"列和"AI级别"标注
- 新增AI贡献度度量指标体系（10个指标），量化AI对过程效率与质量的影响
- 设计四象限AI贡献度分析图，可视化不同AI应用场景的成熟度与价值

(2) 全生命周期活动流程图编制

独立完成了文档中全部7个Mermaid流程图的设计与绘制：

流程图	类型	内容
软件全生命周期总览（AI增强版）	flowchart LR	四大过程组 + 技术过程组全流程 + AI节点标注
需求分析阶段（AI增强）	flowchart TD	业务分析→需求识别→需求定义 + 4个AI参与节点
系统设计阶段（AI增强）	flowchart TD	架构定义→设计定义 + 5个AI参与节点
编码实现阶段（AI增强）	flowchart TD	三子系统分叉→代码安全分级→AI审查 + 5个AI参与节点
测试验证阶段（AI增强）	flowchart TD	CI触发→单元测试→HIL测试→集成测试 + 3个AI参与节点
运维监控阶段（AI增强）	flowchart TD	系统上线→异常检测→根因分析→自动修复 + 3个AI参与节点
角色协作关系图	graph TD	12个角色（含4个AI新角色）的协作网络

每个流程图的设计原则：

- AI参与节点使用虚线连接 + 🤖 emoji标注，视觉上与传统活动明确区分
- 所有AI输出均标注"人工审核"反馈回路，体现"人机协同"的过程设计理念
- 关键决策节点使用菱形判断框，异常路径使用红色标注

(3) 完整角色职责定义

在ISO/IEC 12207原有角色基础上，新增4个AI相关角色并定义了完整的职责、技能要求、过程参与节点：

- **AI训练师**：Prompt工程管理、领域知识库建设、团队AI培训

- **AI测试分析师**：AI测试工具管理、AI生成测试用例质量评估
- **AI质量守门员**：全流程AI产出审查、质量基线与趋势维护
- **AI工具管理员**：工具选型部署、API配额与成本管理、安全合规

同时重新定义了8个传统角色在AI增强后的新职责，并绘制了角色协作关系Mermaid图。

(4) 全流程交付物清单（含AI产出特殊质量要求）

编制了覆盖6个阶段、20+项交付物的完整清单，每项标注：

- 格式要求（Markdown/Mermaid/OpenAPI 3.0/Postman Collection等）
- AI参与角色（生成/审核）
- 质量标准（含量化阈值，如"歧义率<5%"、"MISRA-C符合率>90%"）
- 责任人

(5) 过程度量指标体系设计

设计了双维度度量体系：

- **传统度量指标**（9个）：覆盖进度、质量、效率、稳定性、安全性
- **AI贡献度指标**（10个，创新设计）：AI代码生成率、AI测试用例生成率、AI审查准确率、AI效率提升比、AI缺陷检出率、AI产出一次通过率、AI成本效益比、团队AI使用率等

每个指标均定义了计算公式、目标值和采集频率。这是我在本文档中最为自豪的创新贡献。

(6) 其他章节

- 项目生命周期模型（3个迭代 + Sprint内AI集成节奏甘特图）
- 裁剪说明（基于ISO/IEC 12207 Annex A，含裁减/合并/强化/新增四类操作）
- 完整的参考文献（12篇，涵盖ISO标准、学术论文、行业报告）

2.1.2 03-AI技术应用验证报告 — 协作贡献

参与撰写了以下部分：

- 4.2节"质量对比"中的"边界值覆盖"和"业务逻辑正确性"两个维度的分析
- 4.3节"缺陷发现能力对比"中的预埋缺陷实验设计和数据分析
- 6节"改进建议"中的Prompt模板库建设和覆盖率反馈循环建议

2.1.3 01/02号文档 — 交叉评审贡献

对01号（场景匹配报告）和02号（过程改进方案）进行了两轮交叉评审，共提出17条修改建议

（其中11条被采纳），主要集中在：

- 工具链选型表中增加"集成方式"和"许可类型"维度

- 风险评估矩阵中补充"AI工具依赖风险"和"合规风险"
- Mermaid流程图中的AI节点标注方式统一

2.2 技术工作

2.2.1 LLM智能问答模块开发

独立开发了 `llm_qa.py` 模块（约170行），核心功能：

技术方案：

- 基于DeepSeek API（deepseek-chat模型）实现RAG（检索增强生成）问答
- System Prompt设计为"超市零售领域专家"角色，注入以下领域知识：
- 超市商品分类体系（生鲜、饮品、零食、日化、烘焙等8大类）
- 常见货架陈列规则（先进先出、分类分区、纵向陈列等）
- 缺货判断标准（连续空位>3个视为缺货、排面占比<50%视为不足等）
- 检测结果上下文注入：将YOLO检测到的商品类别、数量、位置信息作为对话上下文
- 支持多轮对话：维护对话历史，实现追问能力（如"可乐还剩多少？建议订多少箱？"）

关键代码片段设计思路：

- `build_context_from_detections()` 函数：将YOLO检测结果转化为LLM可理解的货架状态文本描述
- `generate_answer()` 函数：组装System Prompt + 货架上下文 + 对话历史 + 用户问题，调用DeepSeek API
- 温度参数设为0.3（低随机性，保证回答的专业一致性和可靠性）

测试验证：

- 设计了12个典型超市巡检场景的问答测试用例
- 验证了LLM对"缺货检测"、"品类分布"、"补货建议"三类问题的回答质量
- 发现并修复了3个问题：商品名称映射错误（coke vs cola）、数量统计遗漏零检测框、对话历史截断策略不当

2.2.2 Gradio交互界面集成

独立开发了 `gradio_ui.py` 模块（约280行），实现了端到端的交互流程：

界面布局设计：

- 左侧面板（40%宽度）：视频上传组件 + 检测控制按钮 + 检测结果展示（YOLO标注帧 + 统计表格）
- 右侧面板（60%宽度）：聊天式问答界面 + CLIP语义检索面板
- 底部状态栏：显示处理进度、帧数统计、特征库状态

交互流程集成：

- 1. 用户上传视频 → 触发YOLO检测 → 实时显示标注帧和检测统计
- 2. 检测完成后自动构建FAISS向量索引
- 3. 用户输入问题 → LLM生成回答并显示在聊天面板
- 4. 支持在聊天中上传图片进行以图搜图（CLIP语义检索）
- 5. 所有异常情况（视频格式不支持、API超时、模型未加载）均有友好提示

技术难点解决：

- Gradio的异步状态管理：使用 `gr.State()` 维护检测结果、FAISS索引、对话历史等跨组件共享状态
- 视频帧的流式处理与UI响应：采用生成器模式（`yield`）实现逐帧更新检测结果
- 大型模型文件的延迟加载：YOLO和CLIP模型在首次使用时才加载，减少启动时间

2.2.3 系统级联调与端到端测试

参与全系统联调，我的具体工作：

- 定义各模块间的接口契约（检测结果JSON格式、CLIP特征向量维度、FAISS索引元数据结构）
- 编写联调测试脚本，验证YOLO→CLIP→FAISS→LLM的完整数据流
- 发现并推动修复了2个跨模块接口bug：YOLO输出格式与CLIP输入格式不一致、FAISS元数据字段命名与LLM期望不匹配

2.3 工作量化总结

工作类别		具体内容	工作量估算
文档主笔		04-改进后的软件过程定义文档	~40小时
文档协作		03-验证报告部分章节	~6小时
文档评审		01/02文档交叉评审	~4小时
编码开发		llm_qa.py + gradio_ui.py	~25小时
联调测试		系统联调 + 端到端测试	~8小时
小组协作		会议参与、进度同步、问题讨论	~12小时
总计			~95小时

三、AI工具在个人小组工作中的实际应用过程与分析

3.1 工具使用概览

在本次小组作业中，我深度使用了以下AI工具：

AI工具	使用场景	使用频率	满意度
Claude 4 (Opus 4.7)	文档撰写、Mermaid图表生成、代码审查、技术方案讨论	每日	★★★★★
GitHub Copilot / Cursor	Python代码生成与补全 (llm_qa.py, gradio_ui.py)	每日	★★★★☆
DeepSeek API	作为项目技术组件的LLM问答后端	开发阶段 密集	★★★★☆

3.2 各场景详细应用分析

3.2.1 场景一：AI辅助过程文档撰写（Claude 4）

应用过程：

04号文档的工作量巨大（最终约480行、9章、7个Mermaid图、10+个复杂表格）。我的工作流程为：

- 框架生成阶段：**向Claude提供ISO/IEC 12207的过程组分类和项目上下文，要求生成文档大纲。Claude在30秒内生成了包含9章的初步框架，我基于此进行了3轮迭代细化
- 内容填充阶段：**逐章节向Claude提供要点和约束条件，生成初稿文本。例如在编写"AI贡献度指标"章节时，我描述了量化AI效能的意图，Claude生成了10个候选指标及其计算公式，我筛选保留了最合理的部分
- Mermaid图生成阶段：**描述流程图的需求（如"绘制编码实现阶段的AI增强流程图，需要体现三个子系统的分叉和代码安全分级"），Claude生成Mermaid代码，我在本地渲染后迭代调整布局
- 交叉校验阶段：**将完成的章节提交给Claude进行自我一致性检查——"本文档中提到的AI级别L2在第3章和第4章的定义是否一致？"

效率数据：

工作项	纯人工估算耗时	AI辅助实际耗时	效率提升
文档大纲设计	4小时	1.5小时	62.5%

工作项	纯人工估算耗时	AI辅助实际耗时	效率提升
7个Mermaid流程图	14小时	4小时	71.4%
10+个复杂表格	8小时	3小时	62.5%
参考文献格式整理	2小时	0.5小时	75%
全文通稿与一致性检查	6小时	2小时	66.7%

关键发现：

- AI在**结构化内容生成**（表格、清单、分类）方面效率最高，几乎可以做到"描述需求→得到成品"
- AI在**Mermaid流程图生成**方面表现优秀，但复杂流程图的布局优化仍需人工迭代
- AI在**一致性检查**方面有独特价值——能够发现人工通读容易忽略的术语不一致、编号错位等问题

3.2.2 场景二：AI辅助Python代码开发（Cursor + Claude 4）

应用过程：

在开发 `llm_qa.py` 和 `gradio_ui.py` 时，我使用了Cursor作为主力IDE（AI代码补全）+ Claude 4作为代码审查和技术顾问。

Gradio UI开发的AI协作：

Gradio是我之前不熟悉的框架。传统方式下，我需要阅读官方文档、查找示例代码、反复试错才能完成界面开发。使用AI后：

1. 使用Claude描述界面需求（"两栏布局，左侧视频上传和检测，右侧问答聊天和检索"），Claude生成Gradio界面骨架代码
2. 在Cursor中继续开发，AI自动补全Gradio组件参数和事件绑定代码
3. 遇到 `gr.State()` 异步状态管理问题时，向Claude描述症状，AI定位到事件绑定顺序错误并给出修复方案

效率数据：

开发任务	纯人工估算	AI辅助实际	效率提升
llm_qa.py（170行）	8小时	3小时	62.5%
gradio_ui.py（280行）	16小时	5小时	68.8%

关键发现：

- AI对**框架API的熟悉程度**远超过人类开发者（特别是Gradio这种中小型框架），大大减少了查阅文档的时间
- AI生成的代码在**风格一致性**上优于人工（变量命名、函数签名、异常处理模式统一）
- 但AI对**业务逻辑的正确性**把关不足——例如，它生成的"商品数量统计"逻辑在零检测框时会报除零错误，需要人工发现并修复

3.2.3 场景三：AI作为项目技术组件（DeepSeek API）

应用过程：

这是最有意思的部分——在我的工作中，AI既是开发工具，也是被开发的对象。LLM问答模块本身就是对DeepSeek API的封装调用。

核心设计挑战：

如何让通用LLM表现出超市零售领域的专业知识？我的方案是：

1. **精心设计的System Prompt**：将超市商品分类、陈列规则、缺货标准等编码为System Prompt
2. **检测结果上下文注入**：将YOLO检测的结构化数据转化为自然语言描述，注入每轮对话
3. **对话历史管理**：维护最近3轮对话历史，支持追问场景

实际效果评估：

- 商品相关问题准确率约85%（如"检测到哪些饮料品牌？"）
- 缺货判断准确率约80%（与人工标注对比）
- 补货建议合理性约75%（需要人工复核数量）

3.3 AI应用经验总结

优势	局限性
结构化内容生成效率极高（表格、流程图、清单）	对特定领域的深层业务逻辑理解不足
代码框架和API调用正确率很高	需要人工校准边界条件和异常处理
交叉一致性检查能力超越人工	可能产生"看似正确但实际错误"的幻觉内容
快速上手陌生技术栈（如Gradio）	无法替代实际运行和调试的验证环节

我的AI使用原则（在此次作业中形成）：

1. **AI生成 + 人工审核**：所有AI产出必须经过人工审核才能成为最终交付物
2. **AI负责广度，人工负责深度**：AI系统枚举场景和可能性，人工聚焦关键决策和业务正确性
3. **遇到边界条件、异常处理、安全相关内容时，降低AI依赖**：这些场景下优先依靠自身判断
4. **将好的AI交互模式固化为Prompt模板**：减少重复描述，提升输出一致性

四、收获与不足

4.1 课程教学角度的收获

(1) 对ISO/IEC 12207标准的深度理解

在撰写04号文档的过程中，我通读了ISO/IEC 12207:2017标准全文及其裁剪指南（Annex A），从最初的"标准=条条框框"的刻板印象转变为"标准=可裁剪的通用框架"的认知。具体收获包括：

- 理解了四大过程组（协议、组织项目使能、技术管理、技术）的逻辑关系——组织使能过程为技术过程提供资源和环境，技术管理过程为技术过程提供控制手段
- 掌握了过程裁剪的方法论——不是机械删减，而是基于项目特征（系统类型、团队规模、安全等级）进行针对性调整
- 理解了标准中"过程"与"活动"与"任务"的三层抽象层级，以及它们之间的可追溯关系

(2) 软件过程改进的系统性思维

从前期作业的"照搬标准定义过程"到本次作业的"基于标准 + AI增强改进过程"，我经历了一个重要的认知跃迁：软件过程改进不是简单的"加一个AI步骤"，而是需要从角色、流程、工具、度量四个维度进行系统性重构。这体现在我设计的04号文档中——AI的引入同时影响了角色定义（新增4个角色）、活动流程（每个阶段增加AI节点）、工具链（8个AI工具的集成架构）、度量体系（10个AI贡献度指标）。

(3) AI时代软件工程的新范式认知

通过本次课程的全程AI使用体验，我深刻感受到软件工程正在从"流程驱动"转向"人机协同"范式。这不仅是工具层面的变化，更是方法论层面的变革——传统的需求-设计-编码-测试-运维线性模型正在被AI增强的、更敏捷、更智能的流程所重构。

4.2 合作过程角度的收获

(1) 跨角色协作的价值

同时参与文档撰写和技术开发的双线工作，让我体会到跨角色视野的独特价值。例如，在撰写"AI辅助编码"活动定义时，我基于自己在Gradio开发中使用Cursor的实际体验来校准效率提升数据（从文献中的55%下调到实际体验的45%），使得过程定义更加务实。如果我只做文档而不写代码，可能会照搬文献数据而脱离实际。

(2) 异步协作 + 集中评审的有效性

小组采用的"日常异步（各自按分配任务推进）+ 每周集中评审"模式在12人团队中运行良好。异步模式保证了个人工作的深度，集中评审保证了交付物的一致性。我从中学习到，大团队文档协作的

关键不是"随时保持同步",而是"在关键节点确保对齐"。

(3) 文档版本管理与可追溯性

在多次迭代修改中, Git + Markdown的版本管理方式展现了优势——每次修改有记录、每个版本可回溯、多人修改可自动合并(大部分情况下)。相比传统Word+邮件的方式,效率提升显著。

4.3 AI工具使用角度的收获

(1) Prompt Engineering的实践体悟

在这次作业中,我经历了从"随意提问"到"精心设计Prompt"的成长过程。一些关键认知:

- **上下文越具体,输出越精准**: 给Claude提供完整的ISO/IEC 12207过程定义作为上下文后,其生成的过程文档质量远高于仅给项目名称的情况
- **分步引导优于一步到位**: 复杂内容(如全流程Mermaid图)分步生成——先框架、再细化、再优化——比一次性要求完整输出效果好得多
- **AI的"角色扮演"有真实效果**: 在System Prompt中设定"你是一位资深的软件过程架构师"比不设定角色产出的内容更专业、更结构化

(2) AI工具的边界意识

最大的收获是学会了识别AI的"能力边界"——知道什么时候信任AI、什么时候必须自己思考:

- **信任AI**: 代码框架生成、API调用、文档格式化、一致性检查
- **自己思考**: 业务逻辑设计、架构决策、安全相关代码、对AI输出的最终判断

(3) AI时代的核心竞争力

这次作业让我认识到, AI时代软件工程师的核心竞争力不再是"记住多少API"或"打字多快",而是:

- **系统设计能力**: AI可以生成代码,但架构决策需要人来拍板
- **质量判断能力**: AI可以生成内容,但判断好坏的标准在人
- **领域理解深度**: AI有广度,但深度理解特定领域(如我在超市零售场景中设计的领域知识Prompt)需要人来积累
- **AI工具驾驭能力**: 高效使用AI本身就是一种需要刻意练习的技能

4.4 对未来职业发展的思考

通过本课程的学习和实践,我对未来职业发展形成以下认识:

1. **"AI-enhanced Software Engineer"是必然方向**: 未来5年内,不善于使用AI工具的软件工程师将面临竞争力下降。本课程的系统性AI使用体验让我提前适应了这一趋势。

2. **过程思维是长期竞争力**：具体的编程语言和框架会过时，但软件过程的系统思维——如何在约束条件下组织团队高效交付高质量软件——是持久的价值。ISO/IEC 12207和过程改进方法论的学习让我建立了这种思维框架。
3. **跨领域复合能力越来越重要**：本课程横跨软件工程、AI技术、商业分析三个领域，这种跨领域视角在未来的工作中将越来越重要。纯技术能力可以被AI替代，但"技术+领域+过程"的复合判断力是AI难以替代的。
4. **持续学习能力 > 存量知识**：AI技术的快速迭代（从2024年的Copilot到2026年的Devin等AI Agent）让我深刻感受到，在AI时代，持续学习和适应新技术的能力远比掌握特定技术栈更重要。

4.5 不足之处与改进方向

不足方面	具体表现	改进措施
时间管理	04号文档的撰写在最后一周集中冲刺，前期时间分配不足	采用"每日定量产出"策略，将大文档拆分为每日可完成的小块
AI依赖度把控	个别章节过度依赖AI生成初稿，导致后期人工重写比例高	先独立写出核心逻辑和关键决策，再用AI辅助扩展和格式化
技术深度	Gradio UI部分对底层实现原理理解不够深入（停留在"能用"层面）	在AI工具辅助下快速上手后，应抽时间深入学习关键框架的核心原理
跨模块沟通	与YOLO检测模块同学在接口定义上出现过一次沟通失误，导致1天返工	接口定义应在编码前进行三方确认（提供方+消费方+架构师），并记录为契约文档
测试覆盖	llm_qa.py的测试用例主要覆盖正常场景，异常场景（API超时、返回格式异常）测试不足	使用AI辅助系统性地枚举异常场景，编写对应的Mock测试用例

五、总结

本次软件过程与管理课程的小组作业是一次高强度的"学以致用"实践。在约95小时的投入中，我完成了04号核心文档的主笔、部分协作文档的撰写、两个Python技术模块的独立开发，以及系统级联调工作。通过全程深度使用AI工具（Claude 4、Cursor、DeepSeek API），我切身体验了AI技术对软件工程过程的真实影响——不仅验证了文献中"编码效率提升45%、文档编写效率提升60%"的数据，更重要的是建立了自己在AI时代软件工程实践中的方法论和判断力。

这份个人总结不仅是对过去8周小组工作的回顾，更是对自身在"AI+软件工程"这一新范式下能力成长的确认。感谢课程提供的理论与实践相结合的学习机会，也感谢小组12名同学的高效协作。

撰写时间：2026年6月29日

使用工具：本总结由作者独立撰写，过程中使用Claude 4辅助结构优化和文字润色